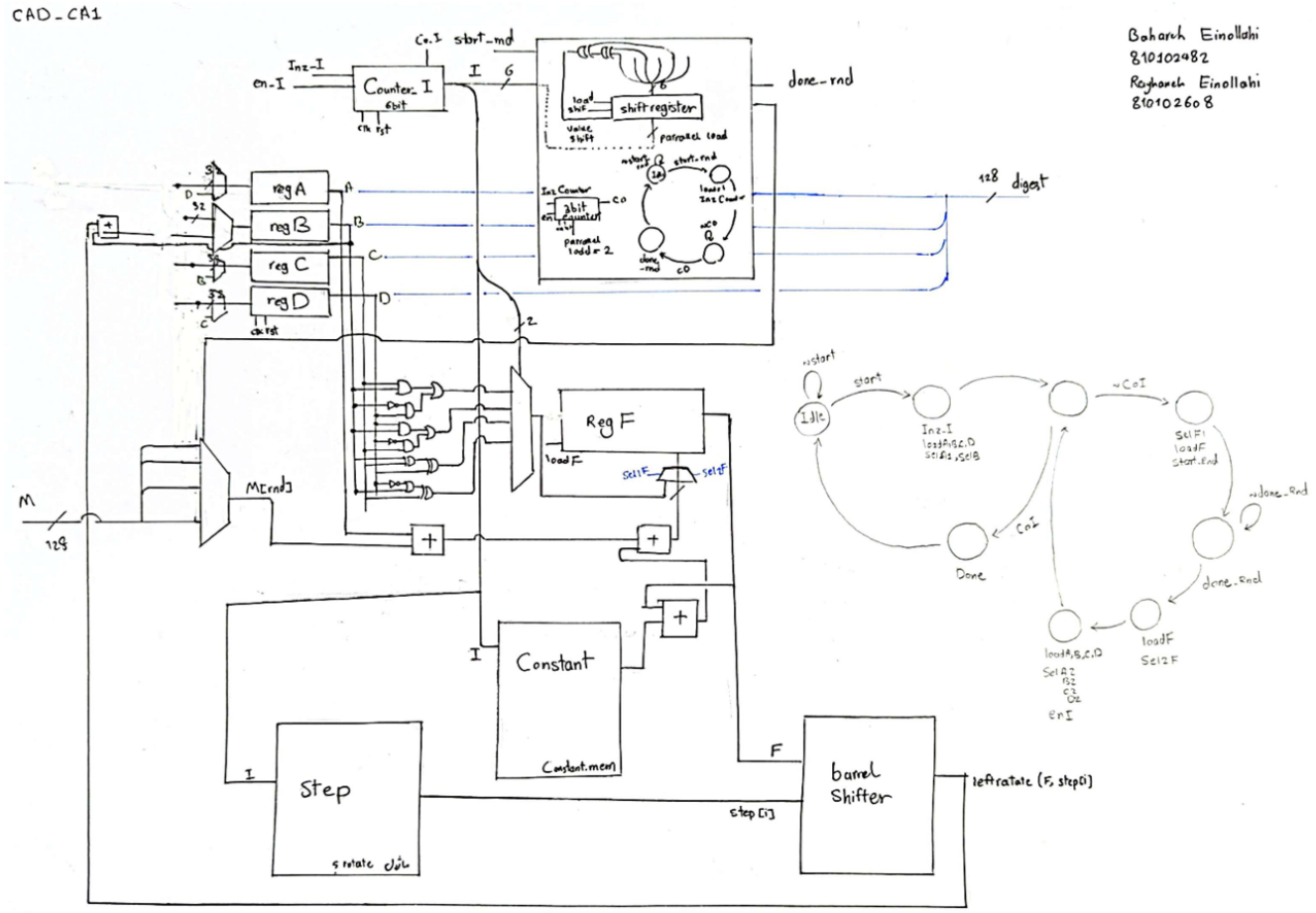
CA01 Generate Sudo Random Numbers

Baharch Einollahi 810102482

Rayhaneh Einollahi 810102608

Data Path



Random generator Controller

```
`timescale 1ns/1ns

module rnd_generator_ct
(
    input clk,
    input rst,
    input start_rnd,
    output logic shift_shreg,
    output logic load_shreg,
    output logic done_rnd
);
    logic inz_counter;
    logic en_counter;
    wire co_counter;
    counter counter(
        .clk(clk),
        .rst(rst),
        .inz(inz_counter),
        .en(en_counter),
        .inz_value(3'b010),
        .cout(co_counter)
    );

    typedef enum logic [1:0] {IDLE, A, B, C} state_t;
    state_t ps, ns;
    always@(posedge clk, posedge rst) begin
        if(rst) begin
            ps <= IDLE;
        end else begin
            ps <= ns;
        end
    end

    always@(*) begin
        case(ps)
            IDLE: begin
                if(start_rnd) begin
                    ns = A;
                end else begin
                    ns = IDLE;
                end
            end
            A: ns = B;
            B: begin
                if(co_counter) begin
                    ns = C;
                end else begin
                    ns = B;
                end
            end
            C: ns = IDLE;
            default: ns = IDLE;
        endcase
    end

    always@(*) begin
        load_shreg = 0;
        shift_shreg = 0;
        inz_counter = 0;
        en_counter = 0;
        done_rnd = 0;
        case(ps)
            IDLE:begin
            end
            A: begin
                load_shreg = 1;
                inz_counter = 1;
            end
            B: begin
                shift_shreg = 1;
                en_counter = 1;
            end
            C: begin
                done_rnd = 1;
            end
            default: begin
            end
        endcase
    end


endmodule
```

Random Generator Datapath

```
module rnd_generator_dp
(
    input clk,
    input rst,
    input load,
    input shift,
    input [5:0] parallel_in,
    output [1:0] rnd_out
);

    wire [5:0] shregout;
    wire input_bit;
    assign input_bit = shregout[5] ^ shregout[3] ^ shregout[1];
    shift_register shreg(clk, rst, load, shift, input_bit, parallel_in, shregout);
    assign rnd_out = shregout[5:4];


endmodule
```

Random Generator top module

```
`timescale 1ns/1ns

module rnd_generator_top
(
    input clk,
    input rst,
    input start_rnd,
    input [5:0] parallel_in,
    output [1:0] rnd_out,
    output done_rnd

);

    wire shift_shreg;
    wire load_shreg;
    rnd_generator_ct ct(
        .clk(clk),
        .rst(rst),
        .start_rnd(start_rnd),
        .shift_shreg(shift_shreg),
        .load_shreg(load_shreg),
        .done_rnd(done_rnd)
    );

    rnd_generator_dp dp(
        .clk(clk),
        .rst(rst),
        .load(load_shreg),
        .shift(shift_shreg),
        .parallel_in(parallel_in),
        .rnd_out(rnd_out)
    );


endmodule
```

Hash Controller

```
`timescale 1ns/1ns

module hash_ct(
    input clk,
    input rst,
    input start,

    input co_counter_i,
    input done_rnd,

    output logic inz_counter_i,
    output logic en_counter_i,
    output logic start_rnd,
    output logic load_m,

    output logic load_a,
    output logic load_b,
    output logic load_c,
    output logic load_d,

    output logic sel_a1,
    output logic sel_b1,
    output logic sel_c1,
    output logic sel_d1,

    output logic sel_a2,
    output logic sel_b2,
    output logic sel_c2,
    output logic sel_d2,

    output logic load_f,
    output logic sel_f1,
    output logic sel_f2,

    output logic done
);



    typedef enum logic [2:0] {IDLE, A, B, C, D, E, F, G} state_t;
    state_t ps, ns;

    always@(posedge clk, posedge rst) begin
        if(rst) begin
            ps <= IDLE;
        end else begin
            ps <= ns;
        end
    end

    always@(*) begin
        case(ps)
            IDLE: begin
                if(start) begin
                    ns = A;
                end else begin
                    ns = IDLE;
                end
            end
            A: ns = B;
            B: begin
                if(co_counter_i) begin
                    ns = G;
                end else begin
                    ns = C;
                end
            end
            C: ns = D;
            D:begin
                if(done_rnd) begin
                    ns = E;
                end else begin
                    ns = D;
                end
            end
            E: ns = F;
            F: ns = B;
            G: ns = IDLE;
            default: ns = IDLE;
        endcase
    end

    always@(*) begin
        inz_counter_i = 0;
        en_counter_i  = 0;
        start_rnd     = 0;
        load_m        = 0;

        load_a = 0;
        load_b = 0;
        load_c = 0;
        load_d = 0;

        sel_a1 = 0;
        sel_b1 = 0;
        sel_c1 = 0;
        sel_d1 = 0;

        sel_a2 = 0;
        sel_b2 = 0;
        sel_c2 = 0;
        sel_d2 = 0;

        load_f = 0;
        sel_f1 = 0;
        sel_f2 = 0;

        done = 0;
        case(ps)
            IDLE:begin
            end
            A: begin
                inz_counter_i = 1;
                load_m        = 1;
                load_a        = 1;
                load_b        = 1;
                load_c        = 1;
                load_d        = 1;

                sel_a1        = 1;
                sel_b1        = 1;
                sel_c1        = 1;
                sel_d1        = 1;

            end
            B:begin end
            C: begin
                sel_f1        = 1;
                load_f        = 1;
                start_rnd     = 1;
            end
            D: begin end
            E: begin
                load_f        = 1;
                sel_f2        = 1;
            end
            F: begin
                load_m        = 1;
                load_a        = 1;
                load_b        = 1;
                load_c        = 1;
                load_d        = 1;

                sel_a2        = 1;
                sel_b2        = 1;
                sel_c2        = 1;
                sel_d2        = 1;

                en_counter_i  = 1;
            end
            G: begin
                done          = 1;
            end
            default: begin
            end
        endcase
    end

endmodule
```

Hash Data Path

```
module hash_dp
(
    input clk,
    input rst,
    input [127:0] parallel_in_m,

    input inz_counter_i,
    input en_counter_i,
    input start_rnd,
    input load_m,

    input load_a,
    input load_b,
    input load_c,
    input load_d,

    input sel_a1,
    input sel_b1,
    input sel_c1,
    input sel_d1,

    input sel_a2,
    input sel_b2,
    input sel_c2,
    input sel_d2,

    input load_f,
    input sel_f1,
    input sel_f2,

    output co_counter_i,
    output done_rnd,
    output [127:0] out
);

    wire [5:0] out_counter_i;
    counter2 #(.WIDTH(6)) counter_i (
        .clk(clk),
        .rst(rst),
        .inz(inz_counter_i),
        .en(en_counter_i),
        .inz_value(0),
        .cout(co_counter_i),
        .out(out_counter_i)
    );
    wire [1:0] rnd_out;
    rnd_generator_top rnd_generator_top(
        .clk(clk),
        .rst(rst),
        .start_rnd(start_rnd),
        .parallel_in(out_counter_i),
        .rnd_out(rnd_out),
        .done_rnd(done_rnd)
    );
    wire [31:0] parallel_in_a;
    wire [31:0] parallel_in_b;
    wire [31:0] parallel_in_c;
    wire [31:0] parallel_in_d;

    wire [31:0] out_a;
    wire [31:0] out_b;
    wire [31:0] out_c;
    wire [31:0] out_d;

    wire [31:0] a0;
    wire [31:0] b0;
    wire [31:0] c0;
    wire [31:0] d0;

    assign a0 = 32'h67452301;
    assign b0 = 32'hefcdab89;
    assign c0 = 32'h98badcfe;
    assign d0 = 32'h10325476;

    wire [31:0] barrel_shifter_out;

    assign parallel_in_a = (sel_a1) ? a0 : (sel_a2)? out_d : 0;
    assign parallel_in_b = (sel_b1) ? b0 : (sel_b2)? out_b + barrel_shifter_out : 0;
    assign parallel_in_c = (sel_c1) ? c0 : (sel_c2)? out_b : 0;
    assign parallel_in_d = (sel_d1) ? d0 : (sel_d2)? out_c : 0;
    register #(32) reg_a (
        .clk(clk),
        .rst(rst),
        .load(load_a),
        .parallel_in(parallel_in_a),
        .data_out(out_a)
    );

    register #(32) reg_b (
        .clk(clk),
        .rst(rst),
        .load(load_b),
        .parallel_in(parallel_in_b),
        .data_out(out_b)
    );

    register #(32) reg_c (
        .clk(clk),
        .rst(rst),
        .load(load_c),
        .parallel_in(parallel_in_c),
        .data_out(out_c)
    );

    register #(32) reg_d (
        .clk(clk),
        .rst(rst),
        .load(load_d),
        .parallel_in(parallel_in_d),
        .data_out(out_d)
    );

    wire [31:0] input_f1, input_f2, parallel_in_f;
    wire [31:0] out_f;
    assign input_f1 =  (out_counter_i[5:4] == 2'b00) ? ((out_b & out_c) | ((~out_b) & out_d)) :
                            (out_counter_i[5:4] == 2'b01) ? ((out_d & out_b) | ((~out_d) & out_c)) :
                            (out_counter_i[5:4] == 2'b10) ? (out_b ^ out_c ^ out_d ) :
                                                            (out_c ^ (out_b | (~out_d)));


    register #(32) reg_f (
        .clk(clk),
        .rst(rst),
        .load(load_f),
        .parallel_in(parallel_in_f),
        .data_out(out_f)
    );

    wire [127:0] out_m;
    register #(128) reg_m (
        .clk(clk),
        .rst(rst),
        .load(load_m),
        .parallel_in(parallel_in_m),
        .data_out(out_m)
    );

    wire [31:0] m_rnd_;
    assign m_rnd_ =       (rnd_out == 2'b11) ? out_m[31:0]   :
                          (rnd_out == 2'b10) ? out_m[63:32]  :
                          (rnd_out == 2'b01) ? out_m[95:64]  :
                          (rnd_out == 2'b00) ? out_m[127:96] :
                          32'b0;

    wire [4:0] step_i_;
    rotate rotate(
        ._i_(out_counter_i),
        .step_i_(step_i_)
    );

    wire [31:0] constant_rom_out;
    constant_rom constant_rom(
        .addr(out_counter_i),
        .data(constant_rom_out)
    );



    barrel_shifter barrel_shifter (
        .value(out_f),
        .shift_amount(step_i_),
        .out(barrel_shifter_out)
    );

    assign input_f2 = (constant_rom_out + out_f) + (m_rnd_ + out_a);
    assign parallel_in_f = (sel_f1) ?  input_f1 : (sel_f2) ? input_f2 : 0;


    assign out = {out_a, out_b, out_c, out_d};

endmodule
```

Hash Top

```
module hash_top(
    input clk,
    input rst,
    input start,
    input [127:0] parallel_in_m,
    output done,
    output [127:0] out
);

    wire co_counter_i;
    wire done_rnd;

    wire inz_counter_i;
    wire en_counter_i;
    wire start_rnd;
    wire load_m;

    wire load_a;
    wire load_b;
    wire load_c;
    wire load_d;

    wire sel_a1;
    wire sel_b1;
    wire sel_c1;
    wire sel_d1;

    wire sel_a2;
    wire sel_b2;
    wire sel_c2;
    wire sel_d2;

    wire load_f;
    wire sel_f1;
    wire sel_f2;

    hash_ct hash_ct(
    .clk(clk),
    .rst(rst),
    .start(start),

    .co_counter_i(co_counter_i),
    .done_rnd(done_rnd),

    .inz_counter_i(inz_counter_i),
    .en_counter_i(en_counter_i),
    .start_rnd(start_rnd),
    .load_m(load_m),

    .load_a(load_a),
    .load_b(load_b),
    .load_c(load_c),
    .load_d(load_d),
    .sel_a1(sel_a1),
    .sel_b1(sel_b1),
    .sel_c1(sel_c1),
    .sel_d1(sel_d1),
    .sel_a2(sel_a2),
    .sel_b2(sel_b2),
    .sel_c2(sel_c2),
    .sel_d2(sel_d2),
    .load_f(load_f),
    .sel_f1(sel_f1),
    .sel_f2(sel_f2),

    .done(done)
    );

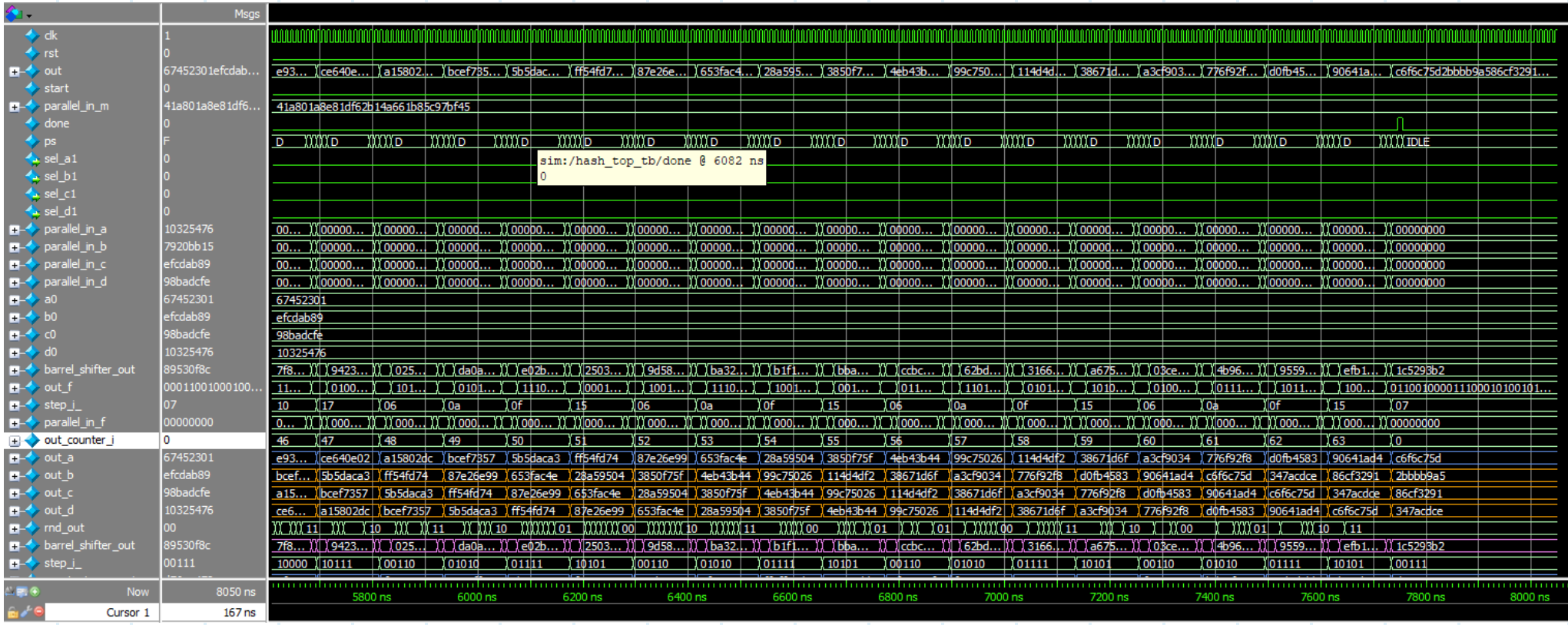

 hash_dp hash_dp(
    .clk(clk),
    .rst(rst),
    .parallel_in_m(parallel_in_m),
    .inz_counter_i(inz_counter_i),
    .en_counter_i(en_counter_i),
    .start_rnd(start_rnd),
    .load_m(load_m),
    .load_a(load_a),
    .load_b(load_b),
    .load_c(load_c),
    .load_d(load_d),
    .sel_a1(sel_a1),
    .sel_b1(sel_b1),
    .sel_c1(sel_c1),
    .sel_d1(sel_d1),
    .sel_a2(sel_a2),
    .sel_b2(sel_b2),
    .sel_c2(sel_c2),
    .sel_d2(sel_d2),
    .load_f(load_f),
    .sel_f1(sel_f1),
    .sel_f2(sel_f2),
    .co_counter_i(co_counter_i),
    .done_rnd(done_rnd),
    .out(out)
    );
```

barrel Shifter

```
module barrel_shifter (
    input  [31:0] value,
    input  [4:0]  shift_amount,
    output [31:0] out
);
    assign out = (value << shift_amount) | (value >> (32 - shift_amount));
endmodule       rayhaneh-einollahi, 4 days ago • add hash dp
```

Barrel Shifter in this logical circuit, barrel shifter is used for shifting based on a value given in input Doing it in combinational circuit.

implementing this using barrelshifter method would be more efficient


Msgs
clk
rst
out
start
parallel_in_m
done
ps
sel_a1
sel_b1
sel_c1
sel_d1
parallel_in_a
parallel_in_b
parallel_in_c
parallel_in_d
a0
b0
c0
d0
barrel_shifter_out
out_f
step_i_
parallel_in_f
out_counter_i
out_a
out_b
out_c
out_d
rnd_out
barrel_shifter_out
step_i_
41a801a8e81df62b14a661b85c97bf45
IDLE
sim:/hash_top_tb/done @ 6082 ns
0
Now
8050 ns
Cursor 1
167 ns
5800 ns
6000 ns
6200 ns
6400 ns
6600 ns
6800 ns
7000 ns
7200 ns
7400 ns
7600 ns
7800 ns
8000 ns